

```

!JSON.stringify(hasVisited).includes(JSON.stringify([x, y + 2]))) {
    unvisitedNeighbors.push(SOUTH);
}
if (x > 1 &&
!JSON.stringify(hasVisited).includes(JSON.stringify([x - 2, y]))) {
    unvisitedNeighbors.push(WEST);
}
if (x < WIDTH - 2 &&
!JSON.stringify(hasVisited).includes(JSON.stringify([x + 2, y]))) {
    unvisitedNeighbors.push(EAST);
}

if (unvisitedNeighbors.length === 0) {
    // BASE CASE
    // All neighboring spaces have been visited, so this is a
    // dead end. Backtrack to an earlier space:
    return;
} else {
    // RECURSIVE CASE
    // Randomly pick an unvisited neighbor to visit:
    let nextIntersection = unvisitedNeighbors[
        Math.floor(Math.random() * unvisitedNeighbors.length)];

    // Move the mark to an unvisited neighboring space:
    let nextX, nextY;
    if (nextIntersection === NORTH) {
        nextX = x;
        nextY = y - 2;
        maze[[x, y - 1]] = EMPTY; // Connecting hallway.
    } else if (nextIntersection === SOUTH) {
        nextX = x;
        nextY = y + 2;
        maze[[x, y + 1]] = EMPTY; // Connecting hallway.
    } else if (nextIntersection === WEST) {
        nextX = x - 2;
        nextY = y;
        maze[[x - 1, y]] = EMPTY; // Connecting hallway.
    } else if (nextIntersection === EAST) {
        nextX = x + 2;
        nextY = y;
        maze[[x + 1, y]] = EMPTY; // Connecting hallway.
    }
    hasVisited.push([nextX, nextY]); // Mark space as visited.
    visit(nextX, nextY); // Recursively visit this space.
}

```